



GITHUB COPILOT

Agent Teams

Coordinating Specialized AI Roles

From solo agents to specialist teams · 20 minutes

The Question

"How do I coordinate multiple specialized agents to handle complex tasks that exceed single-agent capacity?"

When tasks require planning, implementation, review, and testing — different cognitive modes that single agents struggle to balance.

Why Single Agents Hit Limits

Complex tasks exceed single-agent capacity

Real development requires planning, coding, review, testing — different cognitive modes that one agent can't balance simultaneously

Context pollution degrades quality

One agent juggling everything accumulates 50-80% of its context window on irrelevant information, reducing output quality by 30-40%

No specialization, no expertise

General-purpose agents: 60-70% effectiveness. Specialists tuned for one mode: 90-95% effectiveness

The Mental Model Shift

✓ Move Toward

Conductor-delegate architecture

Single responsibility per agent

Tool constraints = role enforcement

Persistent agent memory

Parallel independent work

● Move Away From

Single agent for everything

Universal tool access

Implicit phase transitions

Raw data returns from subagents

Sequential when parallel possible

When to Use Agent Teams

✓ Use Agent Teams When

3+ cognitive modes needed (plan/code/review)

Context window hits 70-80%+ before implementation

Separation of concerns matters

Knowledge should compound across sessions

✗ Don't Use When

Simple task (<200 LOC) → use Plan Mode

Sequential phases only → multi-step-tasks

Branch-level parallelism → parallel-execution

<5 agent tasks/week → overhead exceeds benefit



The Coordinator Pattern

How agent teams actually work under the hood

The Coordinator Pattern

COORDINATOR

Interprets requests → routes via table → spawns specialists → validates → aggregates

NEVER implements directly

↓ delegates to isolated subagents ↓

Lead

Scope, review

Frontend

UI, React

Backend

API, DB

Tester

Tests, QA

Scribe

Memory

Each specialist in **isolated context window** — 6.6% coordinator overhead, 93%+ available for work

Focused Context

Each agent gets only what it needs. No accumulated context from other domains.

Right Tools

Tool constraints = role enforcement. Planners can't edit; implementers can't over-research.

Why This Works

Focused Context

Each agent gets only what it needs. Lead at 1.7%, Frontend at 1.2%, Backend at 1.9% of 200K window. No cross-domain noise.

Parallel Execution

"Team, build the login page" → all agents start simultaneously. 3-5x throughput on parallelizable work.

Quality Checkpoints

Rejected work must be revised by a *different* agent. No self-review loops. 40-60% less rework.

Persistent Memory

Agents write learnings after every session. Week 4 agents know your conventions, patterns, preferences.

Specialization Advantage

Backend agent tuned for APIs, auth, data modeling outperforms generalist by 20-30%. Team of specialists beats single generalist every time.



Squad

A persistent AI development team that lives in your repo

What Is Squad?

github.com/bradygaster/squad

Squad gives you an AI development team through GitHub Copilot. Describe what you're building, and Squad proposes specialists — lead, frontend, backend, tester — that **persist as files in your repo**. Each agent runs in its own context window, reads its own history, and writes back what it learned. **Knowledge compounds across sessions.**

✅ **Not a Chatbot Wearing Hats**

Each team member is spawned as a real subagent with its own tools, memory, and expertise.

✅ **Production-Ready Today**

Install in <10 minutes. Full team with routing, memory, ceremonies, and GitHub Issues integration.

Squad Architecture

"Team, build login page" ← You



SQUAD COORDINATOR

- Routes via `routing.md`
- Spawns agents in parallel
- Enforces reviewer protocol
- Cost-first model selection



Lead
Scope

FE
UI

BE
APIs

Test
QA

Own
Context

Own
Context

Own
Context

Own
Context



● SHARED MEMORY

- `decisions.md` (all read)

Squad Memory System

Layer	What	Who Writes	Who Reads
<code>charter.md</code>	Identity, expertise, voice	Squad (at init)	The agent itself
<code>history.md</code>	Project-specific learnings	Each agent	That agent only
<code>decisions.md</code>	Team-wide decisions	Any agent (via inbox)	All agents
<code>skills/</code>	Reusable patterns (earned)	Agents from real work	All agents
<code>log/</code>	Session history	Scribe	Anyone (searchable)

💡 How Knowledge Compounds Over Time

🌱 Week 1

Project description, tech stack, user name

🌿 Week 4

Conventions, patterns, API design, test strategies

🌳 Week 12

Full architecture, tech debt, regressions, performance

Key Squad Innovations

Per-Agent Model Selection (Cost-First)

Writing code → Claude Sonnet 4.5 (Standard). Non-code work → Claude Haiku 4.5 (Fast). Visual/design → Claude Opus 4.5 (Premium).

Reviewer Protocol — No Self-Review Loops

Agent A submits work → Reviewer rejects → **Agent A locked out** → Agent B must revise. Prevents circular "fixing" loops.

Ralph — Autonomous Work Monitor

Continuously checks for pending work — open issues, draft PRs, review feedback, CI failures. Team never sits idle.

Ceremonies — Structured Team Meetings

Design Review: Before multi-agent tasks — each agent's perspective on interfaces, risks, contracts. **Retrospective:** After failures — focused root-cause analysis.



Getting Started

From zero to a working AI team in under 10 minutes

Three Steps to Your First Team

1

Install

```
mkdir my-project && cd my-project  
git init  
npx github:bradygaster/squad
```

Copies `squad.agent.md` into
`.github/agents/`

2

Form Your Team

```
"I'm building a recipe sharing  
app with React and Node.  
Set up the team."
```

Squad proposes specialists with charters

3

Put Them to Work

```
"Team, build the login page"  
  
"Ripley, fix error handling"
```

All relevant agents launch in parallel

Commit the `.ai-team/` folder. Anyone who clones your repo gets the team — with all accumulated knowledge.

Squad File Structure

```
.ai-team/  
├── team.md  
├── routing.md  
├── ceremonies.md  
├── decisions.md  
├── decisions/inbox/  
├── agents/  
│   ├── {name}/  
│   │   ├── charter.md  
│   │   └── history.md  
├── skills/  
└── log/
```

team.md

Roster — who's on the team

routing.md

Who handles what (routing table)

charter.md

Identity, expertise, boundaries, voice

history.md

What agent knows about YOUR project

Key Takeaways

1. **Single agents have limits** — complex tasks need planning, coding, review, and testing as separate cognitive modes
2. **Conductor-delegate pattern** — coordinator routes, specialists execute in isolated contexts, 3-5x throughput
3. **Squad makes it real** — persistent AI teams that live in your repo and grow smarter every session
4. **Start today** — `npx github:bradygaster/squad` — zero to working team in 10 minutes

Questions?

Try it now: `npx github:bradygaster/squad`

Repo: github.com/bradygaster/squad

Deep dive: [Agent Teams Tech Talk](#) (45 min version)
